

Java, etc. How much are the NLP/ML techniques applicable directly to source code? To answer that question, we need to first identify what are the similarities and differences between source code and natural language? Or “how natural” is the source code in terms of human language characteristics?

Let us recall the fact that NLP techniques are built on the foundation of distributional hypothesis. The meaning of words in a text is inferred from the context. This is known as Firth’s hypothesis, which states that “a word is characterised by the company it keeps.” Essentially, this translates in lay terms to the fact that words that are used in similar contexts have similar meanings. An example is given below.

S1: I went with my mother for her medical consultation.

S2: I accompanied my mother for her medical consultation.

Since the contexts are similar in S1 and S2, we can infer that the words ‘accompanied’ and ‘went with’ have similar meanings. Distributional hypothesis forms the basis for contextual embeddings and contextual representations that form the core of NLP, namely, word embeddings/sentence embeddings, etc. Given this significance, we need to know whether distributional hypothesis would hold for source code as well.

The key difference between natural language text and source code is the fact that natural language is used for communication between two humans. On the other hand, source code is used as the form of communication in two different channels. The first channel is between the software developer who writes the source code and other developers who read/maintain/modify that source code. This is human to human communication like natural language text. On the other hand, source code is used as a means of specifying the computations that need to be performed on a computer. Hence, the second channel of communication is between the software writer and the machine on which the code is intended to be executed. Understanding the meaning of and inferring intent from source code in case of human to human communication is similar in spirit to that of natural language text. Hence, statistical properties that permit standard NLP techniques are applicable to source code as well. However, given that source code is intended to communicate meaning/instructions to a machine, it uses a more precise/non-ambiguous form of communication, and hence is less surprising compared to human communication text. These two properties make it feasible for source code to be analysed and subjected to the same set of NLP techniques that are applicable to natural language text.

If classical NLP techniques can be applied to source code, how do we model source code? Traditional methods of modelling natural language text include n-gram language models, where a word is predicted based on the context. In case of source code, we can identify three different types of models. Representational models of source code are similar to traditional language models. They can be used to predict tokens/code sequences and properties of source code.

Code generation models of source code are used to generate new source code, taking as input either natural language text or source code in some other programming language, or a combination of both. Consider a tool that can automatically translate C code to Python code. Or a tool, which given the natural language description of a function, can generate the source code for that function in a high-level programming language. These are based on code generation models of source code.

Pattern mining models of source code are used to infer latent properties/structure within the source code. They are similar to cluster techniques applied on natural language text, to mine underlying patterns. These models can be used in source code analysis tools, bug prediction tools, etc.

We can also classify source code models based on the type of input and output. Input can be source code in one high level programming language and output can be another high level programming language. A trans-compiler that translates source code in one language to another PL is an example of this. Or, input can be natural language text and output can be source code. Input can also be a combination of natural language text and source code, and output can be source code. Any tool that can automatically generate source code from specifications along with an optional code skeleton in natural language text is an example of this. Or, input can be source code and output can be natural language text. A tool that automatically creates comments/documentation for source code is an example of this.

We can also differentiate the source code models based on the type of machine learning technique used. It can be a supervised learning, unsupervised learning or reinforcement learning based model. We can have DL models of source code that are either sequence to sequence models, transformers or graph neural networks. Different types of source code models are used for different applications such as source code analysis tools, compiler optimisations and automatic code generation. We will see examples of each of these types of models in next month’s column.

There is an excellent paper that discusses how NLP/ML techniques can be used for source code modelling. Titled ‘A survey of machine learning for big code and naturalness’, it is available at <https://arxiv.org/pdf/1709.06182>. I would encourage our readers to take a look at this paper.

Feel free to reach out to me over LinkedIn/email if you need any help in your coding interview preparation. If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Wishing all our readers happy coding until next month! Stay healthy and stay safe. **END** 🐧

By: Sandya Mannarswamy

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.